React Scenario-Based Questions

- A user types in a search box while a huge table is rendering. How can React keep typing smooth?
- Multiple `setState` calls happen in one event. How many renders occur?
- An API response updates five different states. How can React optimize this?
- A page freezes while rendering a large chart. How would concurrent rendering help?
- A slow component blocks button clicks. How would you improve responsiveness
- Your API is called infinitely. What could be wrong?
- A timer keeps using an old state value. Why?
- An event listener always logs outdated data. Why?
- You navigate away from a page, but an API request still updates state. How would you fix it?
- A WebSocket connection remains active after leaving the page. What should you do?
- Your effect depends on an object that changes every render. How do you solve it?
- Why does adding a function to the dependency array cause repeated execution?
- A child wrapped with `React.memo` still re-renders. Why?
- You used `useMemo` everywhere, but performance became worse. Why?
- A callback prop causes unnecessary renders. How would you fix it?

- Expensive calculations execute on every render. How would you optimize them?
- A dashboard contains many charts. Which optimization techniques would you apply?
- When would removing `useMemo` actually improve performance?
- A parent component re-renders, but none of its props changed. Will the child re-render?
- A state update doesn't change the UI. Will React still perform reconciliation?
- Updating one input causes 100 components to render. Why?
- How would you verify whether React actually updated the DOM?
- A component re-renders, but the browser doesn't repaint. Why?
- Why can React re-render without changing the Real DOM?
- A list of 5,000 items updates every second. How does React efficiently update the UI?
- You change one item in a large list, but the whole list appears to re-render. How would you investigate?
- Two sibling components update at different priorities. How does Fiber handle this?
- You accidentally use random keys (`Math.random()`) in a list. What happens internally?
- Your application freezes while rendering a large dashboard. How can Fiber help solve this?
- Why does changing keys cause components to lose their state?
- A component is removed and added back with a different key. What happens to its state?
- A search input filters 50,000 records. How would you keep typing responsive?
- A dashboard performs expensive filtering. How would you prioritize updates?
- A chart takes two seconds to render. How can you keep the UI interactive?

- Which updates should be wrapped inside `startTransition`?
- When would `useDeferredValue` be a better choice than debouncing?
- A page contains three independent API sections. How would you use Suspense?
- One slow component blocks the entire page. How would Suspense help?
- Where would you place Suspense boundaries in a dashboard?
- A lazy-loaded component fails to load. How should the UI behave?
- How can Suspense improve the perceived loading experience?
- Why is a single Suspense boundary around the whole app usually not ideal?

- You notice a memory leak in a production SPA—how do you identify and fix it?
- A component breaks when upgrading a library version—how do you manage dependencies?
- How would you debug a performance bottleneck in a React app using DevTools?
- You need to migrate a legacy frontend codebase to a modern framework—what's your plan?
- How do you ensure secure handling of sensitive user data on the client side?
- Users report intermittent UI glitches in different browsers—how would you troubleshoot?
- A critical UI feature is failing during peak traffic—how do you mitigate the issue?
- How would you implement a robust frontend monitoring and logging system?
- You need to render a large dataset without blocking the main thread—how do you approach it?
- How would you implement A/B testing without affecting current users?
- A CSS animation is janky on mobile devices—how do you identify and fix the issue?

- How do you handle real-time updates in a React application efficiently?
- Your React application suddenly becomes slow after adding new features. How would you identify and fix the issue?
- A page takes 8–10 seconds to load. How would you optimize it?
- Your dashboard has 100+ components. Updating one widget causes the entire dashboard to re-render. How would you solve it?
- A parent component re-renders frequently, causing all child components to re-render. How would you optimize it?
- Your React application consumes too much memory after prolonged usage. How would you debug it?
- How would you profile a React application's performance?
- How would you identify unnecessary re-renders?
- A component performs expensive calculations on every render. How would you optimize it?
- A search input becomes sluggish when filtering a large dataset. How would you fix it?
- How would you optimize rendering for a list containing 100,000 items?
- What does “scalable web applications” mean in your experience **(Scalability)**
- What challenges have you faced when building for large user traffic? **(Scalability / Real-world)**
- Your React app suddenly becomes slow after adding new features. How do you find and fix the issue? **(Debugging Performance)**
- How do you optimize performance of an application? **(General Optimization)**
- What is lazy loading? **(Code Splitting)**
- How do you optimize large bundle size in a React application? **(Bundle Optimization)**
- How do you reduce bundle size in a frontend application? **(Bundle Optimization)**

- What is Tree Shaking? **(Build Optimization)**
- what is memory leak **(Performance Issue)**
- what is different between repaint and reflow **(Rendering Performance)**
- what are core web vitals **(Performance Metrics)**
- explain the performance metrics all questions classify into categgoies **(Performance Metrics Deep Dive)**
- How do you handle caching in React?